DP Computer Science: A 4.3.9 Convolutional Neural Networks (CNNs)

---

1. Core Concepts & Learning Objectives

Learning Intentions

By the end of this lesson, students will be able to:

- Explain why a standard **Multi-Layer Perceptron (MLP)** is not ideal for image recognition tasks
- Describe how CNNs learn a **spatial hierarchy of features**, from simple edges to complex objects
- Outline the basic architecture of a CNN, identifying the function of the **input layer, convolutional layers, pooling layers, fully connected layers, and the output layer**
- Describe the role of a **kernel (filter)** and **stride** in a convolutional layer

---

Key Vocabulary

Term	Definition
CNN (Convolutional Neural Network)	A neural network architecture designed for processing grid-like data, such as images
Convolutional layer	A layer that applies kernels (filters) to the input to detect features
Kernel (filter)	A small matrix that slides over the input to detect specific features
Stride	The step size at which the kernel moves across the input
Feature map	The output of a convolutional layer; highlights where features are detected

Term	Definition
Pooling layer	A layer that downsamples the feature map to reduce dimensionality
Max pooling	A pooling technique that takes the maximum value in each region
Fully connected layer	The final decision-making layer that classifies the extracted features
Spatial hierarchy	The learning of features from simple (edges) to complex (objects)

Approaches to Learning (ATL) Elements

Skill	Description
Thinking (Abstraction & Conceptual Understanding)	Understanding the abstract process of feature extraction through convolution and pooling; grasping the concept of a spatial hierarchy
Communication	Using diagrams and analogies to explain the complex, multi-stage process of a CNN

---

2. Introduction: Why MLPs Fail at Image Recognition

The Problem with Flattening Images

> **A standard MLP requires a 1D vector input. To use an MLP for images, we must "flatten" the image, which destroys all spatial information.**

```text
┌─────────────────────────────────────────────────────────────────┐
│              FLATTENING AN IMAGE                              │
```

```
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│    ┌──────────────────────────────────────────────────────────┐  │
│    │                                                          │  │
│    │    ┌───┬───┬───┐                                        │  │
│    │    │ A │ B │ C │    ───Flatten───▶   A B C D E F       │  │
│    │    ├───┼───┼───┤                     G H I J K L      │  │
│    │    │ D │ E │ F │                     M N O P Q R      │  │
│    │    ├───┼───┼───┤                                        │  │
│    │    │ G │ H │ I │                                        │  │
│    │    └───┴───┴───┘                                        │  │
│    │                                                          │  │
│    │    The spatial relationships between pixels are lost!  │  │
│    │    Neighbouring pixels become separated in the vector. │  │
│    └──────────────────────────────────────────────────────────┘  │
│                                                                  │
│    ❌ MLP cannot see that A and B are next to each other       │
│    ❌ MLP treats all pixels independently                      │
│    ❌ MLP is not suitable for image recognition                │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

How Humans See vs. How CNNs See

> *"When you see a cat, you see edges, then shapes like ears and whiskers, then you combine them. CNNs learn to see in the same way—building a hierarchy of features."*

---

3. CNN Architecture Overview

The Complete CNN Architecture

```text
┌──────────────────────────────────────────────────────────────────┐
│                    CNN ARCHITECTURE                             │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│    ┌──────────────────────────────────────────────────────────┐  │
│    │                    INPUT LAYER                          │  │
```

```
│     │              (Raw image pixels)                         │  │
│     └─────────────────────────┬─────────────────────────────────┘  │
│                               │                                    │
│                               ▼                                    │
│     ┌─────────────────────────────────────────────────────────┐    │
│     │                  CONVOLUTIONAL LAYER                    │  │
│     │          (Detects features: edges, corners)             │  │
│     └─────────────────────────┬─────────────────────────────────┘  │
│                               │                                    │
│                               ▼                                    │
│     ┌─────────────────────────────────────────────────────────┐    │
│     │                 ACTIVATION LAYER                        │  │
│     │            (Adds non-linearity: ReLU)                     │  │
│     └─────────────────────────┬─────────────────────────────────┘  │
│                               │                                    │
│                               ▼                                    │
│     ┌─────────────────────────────────────────────────────────┐    │
│     │                    POOLING LAYER                        │  │
│     │            (Downsamples: reduces size)                    │  │
│     └─────────────────────────┬─────────────────────────────────┘  │
│                               │                                    │
│                               ▼                                    │
│     ┌─────────────────────────────────────────────────────────┐    │
│     │           (More Conv + Pool Layers repeated)              │  │
│     └─────────────────────────┬─────────────────────────────────┘  │
│                               │                                    │
│                               ▼                                    │
│     ┌─────────────────────────────────────────────────────────┐    │
│     │                 FULLY CONNECTED LAYER                   │  │
│     │          (Decision-maker: final classification)           │  │
│     └─────────────────────────┬─────────────────────────────────┘  │
│                               │                                    │
│                               ▼                                    │
│     ┌─────────────────────────────────────────────────────────┐    │
│     │                     OUTPUT LAYER                        │  │
│     │              (Final prediction)                           │  │
│     └─────────────────────────────────────────────────────────┘    │
│                                                                    │
└────────────────────────────────────────────────────────────────────┘
```

4. The Convolutional Layer

What is Convolution?

> Convolution is the process of sliding a small filter (kernel) over an image to detect specific features.

The Flashlight Analogy

> *"A kernel is like a flashlight that shines on a small part of the image at a time. It looks for a specific pattern. As it slides across the image (stride), it highlights where that pattern appears."*

Kernel (Filter) and Stride

```text
┌─────────────────────────────────────────────────────────────────┐
│              KERNEL AND STRIDE                                 │
├─────────────────────────────────────────────────────────────────┤
│                                                                 │
│    ┌─────────────────────────────────────────────────────────┐  │
│    │                   KERNEL (FILTER)                      │  │
│    │                                                         │  │
│    │    ┌─────────────────────────────────────────────────┐  │  │
│    │    │               3x3 Matrix                       │  │  │
│    │    ├───────────────┬───────────────┬─────────────────┤  │  │
│    │    │       1       │       0       │       1         │  │  │
│    │    ├───────────────┼───────────────┼─────────────────┤  │  │
│    │    │       0       │       1       │       0         │  │  │
│    │    ├───────────────┼───────────────┼─────────────────┤  │  │
│    │    │       1       │       0       │       1         │  │  │
│    │    └───────────────┴───────────────┴─────────────────┘  │  │
│    │                                                         │  │
│    │    Designed to detect a specific pattern               │  │
│    │    (e.g., vertical edge, horizontal edge, corner)     │  │
│    └─────────────────────────────────────────────────────────┘  │
│                                                                 │
│    ┌─────────────────────────────────────────────────────────┐  │
│    │                   STRIDE                               │  │
│    │                                                         │  │
│    │    The step size at which the kernel moves              │  │
│    │                                                         │  │
```

```
│    │    Stride = 1: Moves 1 pixel at a time              │  │
│    │    Stride = 2: Moves 2 pixels at a time            │  │
│    │                                                     │  │
│    │    Larger stride = smaller output feature map       │  │
│    └─────────────────────────────────────────────────────┘  │
│                                                            │
└────────────────────────────────────────────────────────────┘
```

How Convolution Works

```text
┌────────────────────────────────────────────────────────────┐
│              CONVOLUTION PROCESS                           │
├────────────────────────────────────────────────────────────┤
│                                                            │
│    Step 1: Place the kernel over a region of the image    │
│                                                            │
│    Step 2: Multiply each image pixel by the corresponding │
│            kernel value                                    │
│                                                            │
│    Step 3: Sum all the products                           │
│                                                            │
│    Step 4: Place the result in the feature map            │
│                                                            │
│    Step 5: Slide the kernel to the next position         │
│            (controlled by stride)                         │
│                                                            │
│    Step 6: Repeat until the entire image is covered       │
│                                                            │
└────────────────────────────────────────────────────────────┘
```

Feature Map

> A feature map is the output of a convolutional layer. It shows where the kernel's pattern was detected in the image—higher values indicate a stronger match.

---

5. Interactive Activity: "Be the Filter"

The Input Image (5×5 Grid)

```text
┌─────────────────────────────────────────┐
│              INPUT IMAGE (5×5)           │
├─────────────────────────────────────────┤
│                                         │
│    ┌───┬───┬───┬───┬───┐                │
│    │ 0 │ 1 │ 0 │ 0 │ 0 │                │
│    ├───┼───┼───┼───┼───┤                │
│    │ 0 │ 1 │ 0 │ 0 │ 0 │                │
│    ├───┼───┼───┼───┼───┤                │
│    │ 1 │ 1 │ 1 │ 1 │ 1 │     (A cross shape)  │
│    ├───┼───┼───┼───┼───┤                │
│    │ 0 │ 1 │ 0 │ 0 │ 0 │                │
│    ├───┼───┼───┼───┼───┤                │
│    │ 0 │ 1 │ 0 │ 0 │ 0 │                │
│    └───┴───┴───┴───┴───┘                │
│                                         │
└─────────────────────────────────────────┘
```

The Kernel (3×3)

```text
┌─────────────────────────────────────────┐
│              KERNEL (3×3)                │
├─────────────────────────────────────────┤
│                                         │
│    ┌───┬───┬───┐                        │
│    │ 0 │ 1 │ 0 │                        │
│    ├───┼───┼───┤       (Designed to detect vertical lines)  │
│    │ 0 │ 1 │ 0 │                        │
│    ├───┼───┼───┤                        │
│    │ 0 │ 1 │ 0 │                        │
│    └───┴───┴───┘                        │
```

```
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Step-by-Step Convolution

Position 1: Top-Left

```text

┌──────────────────────────────────────────────────────────────────┐
│              POSITION 1                                         │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│    Image Patch (3×3)      │   Kernel (3×3)     │  Calculation  │
│    ┌───┬───┬───┐         │   ┌───┬───┬───┐   │               │
│    │ 0 │ 1 │ 0 │         │   │ 0 │ 1 │ 0 │   │  (0×0)=0      │
│    ├───┼───┼───┤    ×    │   ├───┼───┼───┤   │  (1×1)=1      │
│    │ 0 │ 1 │ 0 │         │   │ 0 │ 1 │ 0 │   │  (0×0)=0      │
│    ├───┼───┼───┤         │   ├───┼───┼───┤   │  (0×0)=0      │
│    │ 1 │ 1 │ 1 │         │   │ 0 │ 1 │ 0 │   │  (1×1)=1      │
│    └───┴───┴───┘         │   └───┴───┴───┘   │  (0×0)=0      │
│                                                │  (1×0)=0      │
│                                                │  (1×1)=1      │
│                                                │  (1×0)=0      │
│                                                │  Sum = 3      │
│                                                                  │
│    Feature Map Value (Top-Left) = 3                             │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Position 2: Top-Middle

```text

┌──────────────────────────────────────────────────────────────────┐
│              POSITION 2                                         │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│    Image Patch (3×3)      │   Kernel (3×3)     │  Calculation  │
│    ┌───┬───┬───┐         │   ┌───┬───┬───┐   │               │
│    │ 1 │ 0 │ 0 │         │   │ 0 │ 1 │ 0 │   │  (1×0)=0      │
```

```
│     ├───┼───┼───┤    ×   │    ├───┼───┼───┤    │  (0×1)=0        │
│     │ 1 │ 0 │ 0 │        │    │ 0 │ 1 │ 0 │    │  (0×0)=0        │
│     ├───┼───┼───┤        │    ├───┼───┼───┤    │  (1×0)=0        │
│     │ 1 │ 1 │ 1 │        │    │ 0 │ 1 │ 0 │    │  (1×1)=1        │
│     └───┴───┴───┘        │    └───┴───┴───┘    │  (1×0)=0        │
│                                                  │  (1×0)=0        │
│                                                  │  (1×1)=1        │
│                                                  │  (1×0)=0        │
│                                                  │  Sum = 2        │
│                                                                      │
│    Feature Map Value (Top-Middle) = 2                              │
│                                                                      │
└──────────────────────────────────────────────────────────────────┘
```

Position 3: Top-Right

Image Patch	Calculation	Sum
`0 0 0`	`(0×0) + (0×1) + (0×0) + (0×0) + (0×1) + (0×0) + (0×0) + (1×1) + (1×0)`	1
`0 0 0`		
`1 1 1`		

Completed Feature Map (3×3)

```text

┌──────────────────────────────────────────────────────────────────┐
│              FEATURE MAP (3×3)                                  │
├──────────────────────────────────────────────────────────────────┤
│                                                                    │
│     ┌───┬───┬───┐                                              │
│     │ 3 │ 2 │ 1 │                                              │
│     ├───┼───┼───┤                                              │
│     │ 3 │ 2 │ 1 │                                              │
│     ├───┼───┼───┤                                              │
│     │ 3 │ 2 │ 1 │                                              │
│     └───┴───┴───┘                                              │
│                                                                    │
```

```
│    High values (3) show where the vertical line was detected  │
│    Medium values (2) show partial matches                     │
│    Low values (1) show weak matches                           │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

What This Means

Value	Interpretation
3	Perfect match—vertical line detected here!
2	Strong match
1	Weak match

> *"The feature map is a 'heat map' showing where the kernel found the pattern. High values indicate strong matches."*

---

6. The Spatial Hierarchy of Features

Learning from Simple to Complex

```text
┌─────────────────────────────────────────────────────────────────┐
│              SPATIAL HIERARCHY OF FEATURES                     │
├─────────────────────────────────────────────────────────────────┤
│                                                                 │
│    ┌───────────────────────────────────────────────────────┐    │
│    │              EARLY LAYERS                            │   │
│    │              Simple Features                          │   │
│    │                                                       │   │
│    │    ┌───┐  ┌───┐  ┌───┐                              │   │
│    │    │   │  │ / │  │   │                              │   │
│    │    │ ─ │  │ \ │  │ ○ │  ← Edges, corners, blobs     │   │
│    │    │   │  │   │  │   │                              │   │
│    │    └───┘  └───┘  └───┘                              │   │
```

```
│     └──────────────────────────────────────────────────────────┘   │
│                               │                                     │
│                               ▼                                     │
│     ┌──────────────────────────────────────────────────────────┐   │
│     │              MIDDLE LAYERS                              │   │
│     │              More Complex Shapes                        │   │
│     │                                                          │   │
│     │    ┌───┐  ┌───┐  ┌───┐                                │   │
│     │    │ █ │  │ █ │  │   │                                │   │
│     │    │ █ │  │███│  │ █ │  ← Eyes, wheels, ears         │   │
│     │    │ █ │  │   │  │ █ │                                │   │
│     │    └───┘  └───┘  └───┘                                │   │
│     └──────────────────────────────────────────────────────────┘   │
│                               │                                     │
│                               ▼                                     │
│     ┌──────────────────────────────────────────────────────────┐   │
│     │              LATER LAYERS                               │   │
│     │              Complete Objects                           │   │
│     │                                                          │   │
│     │    ┌───────┐  ┌───────┐                               │   │
│     │    │  🐱   │  │  🚗   │  ← Faces, animals, cars       │   │
│     │    │       │  │       │                               │   │
│     │    └───────┘  └───────┘                               │   │
│     └──────────────────────────────────────────────────────────┘   │
│                                                                     │
└─────────────────────────────────────────────────────────────────────┘
```

The Analogy: The Assembly Line

> *"The Conv/Pool layers are like specialised workers on an assembly line. The first worker spots simple lines. The next worker takes those lines and spots corners. The next worker takes the corners and spots squares, and so on. The Fully Connected layer is the final quality control inspector who looks at all the assembled parts and makes the final call: 'This is a cat.'"*

---

7. The Pooling Layer

What is Pooling?

> Pooling is a downsampling technique that reduces the size of the feature map, making the model more efficient and robust to small changes.

Max Pooling

```text
┌──────────────────────────────────────────────────────────────────┐
│              MAX POOLING (2×2, Stride 2)                        │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│    Input Feature Map (4×4)              Output (2×2)           │
│                                                                  │
│    ┌───┬───┬───┬───┐                   ┌───┬───┐               │
│    │ 3 │ 1 │ 1 │ 2 │                   │ 3 │ 4 │               │
│    ├───┼───┼───┼───┤       Max         ├───┼───┤               │
│    │ 5 │ 2 │ 4 │ 0 │     Pooling       │ 8 │ 6 │               │
│    ├───┼───┼───┼───┤                   └───┴───┘               │
│    │ 7 │ 8 │ 1 │ 3 │                                            │
│    ├───┼───┼───┼───┤                                            │
│    │ 0 │ 2 │ 6 │ 5 │                                            │
│    └───┴───┴───┴───┘                                            │
│                                                                  │
│    Each 2×2 region is reduced to its maximum value             │
│    This preserves the strongest features while reducing size   │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Why Use Pooling?

Reason	Explanation
Reduce Dimensionality	Smaller feature maps = less computation
Improve Robustness	Small shifts in position don't affect the output
Prevent Overfitting	Reduces the number of parameters
Abstract Features	Focuses on the most important features

8. The Fully Connected Layer

What is the Fully Connected Layer?

> The Fully Connected (FC) layer is the final decision-maker. It takes the high-level features extracted by the convolutional layers and makes the final classification.

```text
┌─────────────────────────────────────────────────────────────────┐
│              FULLY CONNECTED LAYER                             │
├─────────────────────────────────────────────────────────────────┤
│                                                                 │
│    ┌─────────────────────────────────────────────────────────┐  │
│    │  Feature Maps (from Conv+Pool)  →  Flatten  →  FC  │  │
│    │                                                         │  │
│    │     ┌───┬───┐          ┌───┐        ┌─────────────┐   │  │
│    │     │ 3 │ 4 │          │ 3 │        │  Cat:  0.9  │   │  │
│    │     ├───┼───┤  ────▶  │ 4 │  ────▶│  Dog:  0.1  │   │  │
│    │     │ 8 │ 6 │          │ 8 │        │  Bird: 0.0  │   │  │
│    │     └───┴───┘          │ 6 │        └─────────────┘   │  │
│    │                        └───┘                            │  │
│    │                                                         │  │
│    │    The FC layer is a standard MLP that makes          │  │
│    │    the final classification decision.                 │  │
│    └─────────────────────────────────────────────────────────┘  │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

---

9. CNN Layers Summary

Layer	Function	Key Concept
Input Layer	Receives raw image pixels	Image dimensions (width, height, channels)
Convolutional Layer	Detects features (edges, corners)	Kernels (filters), stride, feature maps

Layer	Function	Key Concept
Activation Layer	Adds non-linearity	ReLU, Sigmoid, etc.
Pooling Layer	Downsamples feature maps	Max pooling, average pooling
Fully Connected Layer	Makes final classification	MLP-like decision-maker
Output Layer	Produces final prediction	Softmax for classification

---

10. Real-World Applications of CNNs

Application	Description
Image Classification	Identify what is in an image (cat, dog, car)
Object Detection	Find and locate objects in an image
Facial Recognition	Identify individuals from face images
Self-Driving Cars	Detect pedestrians, traffic signs, and other vehicles
Medical Imaging	Detect diseases in X-rays and MRIs
Video Analysis	Action recognition, surveillance

---

11. Lesson Sequence

Lesson Plan (60 minutes)

Phase	Time	Activity
Recap & Hook	10 min	Recap MLPs; discuss flattening problem; show cat image

Phase	Time	Activity
Lecture: Convolutional Layer	15 min	Flashlight analogy; kernel, stride, feature map
Interactive Activity	15 min	"Be the Filter" — manual convolution on 5x5 grid
Lecture: Pooling & Architecture	15 min	Max pooling; full CNN architecture; spatial hierarchy
Consolidation	5 min	Recap; real-world applications

Discussion Questions

Question	Purpose
"Why is an MLP not suitable for image recognition?"	Understand the flattening problem
"What is the role of the kernel in a CNN?"	Understand convolution
"What does the feature map show?"	Understand output of convolution
"Why do we use pooling layers?"	Understand downsampling
"How does a CNN learn a hierarchy of features?"	Understand spatial hierarchy

---

12. Assessment Activities

Assessment Type	Description
Class Participation	Observe engagement and contributions

Assessment Type	Description
"Be the Filter" Activity	Assess understanding of convolution
Exit Ticket	Gauge understanding and identify areas needing clarification

Sample Exit Ticket Questions:

1. What is the problem with using an MLP for image recognition?
2. What is the role of a kernel in a CNN?
3. What is max pooling and why is it used?
4. What is the purpose of the fully connected layer in a CNN?

---

13. Differentiation

Level	Strategy
Support	Use the "Flashlight" analogy; step through the "Be the Filter" activity carefully
Challenge	Explore different kernel sizes and strides; research CNN architectures (LeNet, AlexNet)

---

14. Resources

- **Presentation** (Slide Deck)
- **eBook** (A4.3.9 with examples)
- **Worksheet:** CNN structure and convolution calculations
- **Worksheet Answers**
- **Quizlet** (A4.3.9 vocabulary flashcards)

---

15. Summary: Key Takeaways

Concept	Key Point
MLP for Images	Not ideal—flattening loses spatial information
CNN	Designed for image recognition
Convolution	Sliding a kernel to detect features
Kernel/Filter	Small matrix that detects specific patterns
Stride	Step size of the kernel
Feature Map	Output of convolution; shows where features are detected
Pooling	Downsamples to reduce size and improve robustness
Fully Connected	Final decision-maker
Spatial Hierarchy	Learns features from simple to complex

```text
┌─────────────────────────────────────────────────────────────────┐
│                    KEY REMINDERS                                │
├─────────────────────────────────────────────────────────────────┤
│                                                                 │
│    ✅ MLP flattens images → loses spatial info                  │
│    ✅ CNN preserves spatial relationships                       │
│    ✅ Kernel = filter that detects features                     │
│    ✅ Stride = step size of the kernel                          │
│    ✅ Feature map = output of convolution                       │
│    ✅ Pooling = downsamples the feature map                     │
│    ✅ Fully Connected = decision-maker                          │
│    ✅ Spatial hierarchy = simple → complex features             │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

> *"Convolutional Neural Networks are designed to see the world like we do—building a hierarchy of features from simple edges to complex objects."*